GHOST VEHICLE DETECTION IN FOG

4-Sensor 360° Connection Guide, Flowchart & TM4C123 Final Code

TM4C123GH6PM + 4 Analog Acoustic/Pressure Sensor Units + SSD1306 OLED + LED + Buzzer + HC-SR04

0. IMPORTANT SENSOR ASSUMPTION

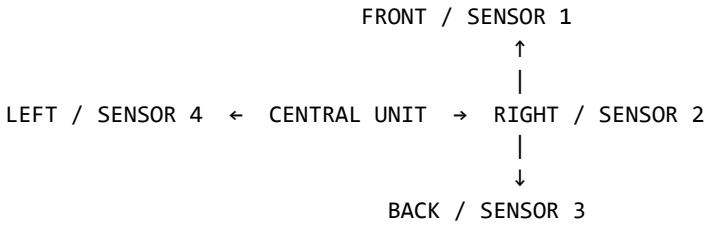
This document assumes that each of the four sensor modules has an analog output pin (OUT/AOUT) that can be connected to a TM4C123 ADC input. If by "MPU" you mean an MPU6050/MPU9250/MPU6500 IMU, do NOT use the analog wiring or code below; those devices use I²C/SPI and require a different design.

1. FINAL 5-BREADBOARD ARRANGEMENT

Use four small breadboards for the four directional sensor units and one central breadboard for the TM4C123 connections, power distribution, OLED, LED, buzzer and HC-SR04.

Breadboard	Position	Main purpose
BB1	Front	Sensor 1 / FRONT
BB2	Right	Sensor 2 / RIGHT
BB3	Back	Sensor 3 / BACK
BB4	Left	Sensor 4 / LEFT
BB5	Center	TM4C123 interface, common power/GND, OLED, LED, buzzer, HC-SR04

Physical concept:



2. COMPLETE PIN CONNECTION TABLE

Hardware	Pin/Terminal	TM4C123 connection	Notes
Front sensor (S1)	VCC	3.3V	Common 3.3V rail
Front sensor (S1)	GND	GND	Common ground
Front sensor (S1)	AOUT/OUT	PE3 / AIN0	12-bit ADC
Right sensor (S2)	VCC	3.3V	Common 3.3V rail
Right sensor (S2)	GND	GND	Common ground
Right sensor (S2)	AOUT/OUT	PE2 / AIN1	12-bit ADC
Back sensor (S3)	VCC	3.3V	Common 3.3V rail
Back sensor (S3)	GND	GND	Common ground
Back sensor (S3)	AOUT/OUT	PE1 / AIN2	12-bit ADC
Left sensor (S4)	VCC	3.3V	Common 3.3V rail
Left sensor (S4)	GND	GND	Common ground
Left sensor (S4)	AOUT/OUT	PE0 / AIN3	12-bit ADC
SSD1306 OLED	VCC	3.3V	I²C OLED
SSD1306 OLED	GND	GND	Common ground
SSD1306 OLED	SDA	PB3	I²C0 SDA
SSD1306 OLED	SCL	PB2	I²C0 SCL
LED	Anode	PB4 through 220–330 Ω	Cathode to GND
LED	Cathode	GND	Common ground
Active-high buzzer	+	PB5	Use suitable driver if current is too high
Active-high buzzer	−	GND	Common ground
HC-SR04	VCC	5V/VBUS	Do not power from 3.3V
HC-SR04	GND	GND	Common ground

HC-SR04	TRIG	PB6	GPIO output
HC-SR04	ECHO	PB7 through divider	5V echo must be level-shifted

3. POWER DISTRIBUTION ON THE CENTRAL BREADBOARD

Use BB5 as the common power hub. Connect TM4C123 3.3V to the 3.3V rail and TM4C123 GND to the ground rail. Distribute these rails to all four sensor breadboards and the OLED. The HC-SR04 uses 5V/VBUS, but its ECHO must be level-shifted before PB7.

All five breadboards must share a common GND. Do not connect two different regulated voltage outputs together.

4. HC-SR04 ECHO PROTECTION

Do NOT connect HC-SR04 ECHO directly to PB7. Use a resistor divider:

```
HC-SR04 ECHO → 1 kΩ → junction → PB7
                         ↓
                       2 kΩ
                         ↓
                        GND
```

This is a nominal divider from 5 V to about 3.33 V. Verify the actual module and resistor values before powering the circuit.

5. 360° WORKING FLOWCHART

START

↓

Initialize GPIO, ADC, I²C, OLED, LED, Buzzer and HC-SR04

↓

Read 4 analog sensor channels

↓

Compare sensor levels

↓

Find strongest sensor response / dominant direction

↓

Measure proximity using HC-SR04

↓

Update OLED

↓

Distance > 30 cm → Monitoring / LED OFF / Buzzer OFF

10–30 cm → Vehicle approaching / LED ON / Buzzer OFF

≤ 10 cm → Critical / LED ON / Buzzer ON

↓

```
Repeat continuously
```

6. SYSTEM WORKING

1. 1. The four sensors are physically arranged around the test environment: Front, Right, Back and Left.
2. 2. Each analog output is sampled by a separate TM4C123 ADC channel.
3. 3. The controller compares the four readings and identifies the strongest response as the dominant source direction.
4. 4. The HC-SR04 provides supporting physical proximity information.
5. 5. The OLED displays the distance, strongest sensor value and warning state.
6. 6. The LED and buzzer provide the warning according to the programmed distance thresholds.
7. 7. The system repeats continuously.

7. SOFTWARE FILE STRUCTURE

Create these files in the Keil project. The code below is written to avoid C99-only declaration syntax.

- main.c
- adc.c / adc.h
- delay.c / delay.h
- i2c.c / i2c.h
- oled.c / oled.h
- font.c / font.h
- led.c / led.h
- buzzer.c / buzzer.h
- ultrasonic.c / ultrasonic.h

8. main.c

```
#include "TM4C123GH6PM.h"
#include "adc.h"
#include "delay.h"
#include "i2c.h"
#include "oled.h"
#include "buzzer.h"
#include "led.h"
#include "ultrasonic.h"

typedef enum
{
    DIR_FRONT = 0,
    DIR_RIGHT = 1,
    DIR_BACK  = 2,
    DIR_LEFT  = 3
} Direction_t;

static Direction_t GetDirection(unsigned short adc[4], unsigned short *maxVal)
{
    unsigned char i;
    unsigned char maxIdx;

    maxIdx = 0;
    for(i=1; i<4; i++)
    {
        if(adc[i] > adc[maxIdx])
            maxIdx = i;
    }

    *maxVal = adc[maxIdx];
    return (Direction_t)maxIdx;
}

static const char *DirectionText(Direction_t d)
{
    if(d == DIR_FRONT) return "FRONT";
    if(d == DIR_RIGHT) return "RIGHT";
```

```
    if(d == DIR_BACK)  return "BACK";
    return "LEFT";
}

static void UpdateAlerts(unsigned short dist)
{
    if(dist == 0xFFFF)
    {
        LED_Off();
        Buzzer_Off();
    }
    else if(dist > 30)
    {
        LED_Off();
        Buzzer_Off();
    }
    else if(dist > 10)
    {
        LED_On();
        Buzzer_Off();
    }
    else
    {
        LED_On();
        Buzzer_On();
    }
}

static const char *StatusText(unsigned short dist)
{
    if(dist == 0xFFFF) return "NO ECHO";
    if(dist > 30) return "MONITOR";
    if(dist > 20) return "DETECTED";
    if(dist > 10) return "APPROACH";
    return "CRITICAL";
}

int main(void)
{
    unsigned short adc[4];
    unsigned short maxVal;
    unsigned short dist;
    Direction_t dir;

    Buzzer_Init();
    Buzzer_Off();

    LED_Init();
    LED_Off();

    ADC0_Init4Ch();
    I2C0_Init();
    OLED_Init();
    Ultrasonic_Init();

    OLED_Clear();
    OLED_SetCursor(0,0);
    OLED_PrintString("GHOST VEHICLE");
    OLED_SetCursor(1,0);
    OLED_PrintString("360 SENSOR READY");
    Delay_ms(1500);

    while(1)
    {
        ADC0_ReadAll(adc);
        dir = GetDirection(adc, &maxVal);
        dist = Ultrasonic_GetDistanceCM();

        OLED_Clear();

        OLED_SetCursor(0,0);
        OLED_PrintString("DIST:");
        if(dist == 0xFFFF)
            OLED_PrintString("NOECHO");
        else
```

```
        {
            OLED_PrintNumber(dist);
            OLED_PrintString("CM");
        }

        OLED_SetCursor(1,0);
        OLED_PrintString("DIR:");
        OLED_PrintString(DirectionText(dir));

        OLED_SetCursor(2,0);
        OLED_PrintString("SND:");
        OLED_PrintNumber(maxVal);

        OLED_SetCursor(3,0);
        OLED_PrintString(StatusText(dist));

        UpdateAlerts(dist);
        Delay_ms(150);
    }
}
```

9. adc.h

```
#ifndef ADC_H
#define ADC_H
#include "TM4C123GH6PM.h"
void ADC0_Init4Ch(void);
void ADC0_ReadAll(unsigned short adc[4]);
#endif
```

10. adc.c

```
#include "TM4C123GH6PM.h"
#include "adc.h"

void ADC0_Init4Ch(void)
{
    SYSCTL->RCGCADC |= 1U;
    SYSCTL->RCGCGPIO |= (1U << 4);       /* Port E */
    while((SYSCTL->PRGPIO & (1U << 4)) == 0);

    GPIOE->AFSEL |= 0x0F;                /* PE0-PE3 analog */
    GPIOE->DEN &= ~0x0F;
    GPIOE->AMSEL |= 0x0F;

    ADC0->ACTSS &= ~(1U << 1);           /* Disable SS1 */
    ADC0->EMUX &= ~(0xFU << 4);          /* Software trigger */
    ADC0->SSMUX1 = (0U << 0) |
                   (1U << 4) |
                   (2U << 8) |
                   (3U << 12);           /* AIN0,A1,A2,A3 */
    ADC0->SSCTL1 = (1U << 0) | (1U << 1) |
                   (1U << 4) | (1U << 5) |
                   (1U << 8) | (1U << 9) |
                   (1U << 12) | (1U << 13);
    ADC0->ACTSS |= (1U << 1);
}

void ADC0_ReadAll(unsigned short adc[4])
{
    ADC0->PSSI = (1U << 1);
    while((ADC0->RIS & (1U << 1)) == 0);

    adc[0] = ADC0->SSFIFO1 & 0x0FFF;
    adc[1] = ADC0->SSFIFO1 & 0x0FFF;
    adc[2] = ADC0->SSFIFO1 & 0x0FFF;
    adc[3] = ADC0->SSFIFO1 & 0x0FFF;

    ADC0->ISC = (1U << 1);
}
```

11. delay.h

```
#ifndef DELAY_H
#define DELAY_H
void Delay_ms(unsigned long ms);
```

```
void Delay_us(unsigned long us);
#endif
```

12. delay.c

```
#include "TM4C123GH6PM.h"
#include "delay.h"

void Delay_ms(unsigned long ms)
{
    unsigned long i, j;
    for(i = 0; i < ms; i++)
        for(j = 0; j < 4000; j++);
}

void Delay_us(unsigned long us)
{
    unsigned long i;
    for(i = 0; i < us * 4; i++);
}
```

13. i2c.h

```
#ifndef I2C_H
#define I2C_H
void I2C0_Init(void);
void I2C0_WriteByte(unsigned char address, unsigned char data);
#endif
```

14. i2c.c

```
#include "TM4C123GH6PM.h"
#include "i2c.h"

static void I2C0_Wait(void)
{
    while(I2C0->MCS & 1U);
}

void I2C0_Init(void)
{
    SYSCTL->RCGCI2C |= 1U;
    SYSCTL->RCGCGPIO |= (1U << 1);
    while((SYSCTL->PRGPIO & (1U << 1)) == 0);

    GPIOB->AFSEL |= (1U << 2) | (1U << 3);
    GPIOB->ODR   |= (1U << 3);            /* SDA open drain */
    GPIOB->DEN   |= (1U << 2) | (1U << 3);
    GPIOB->PCTL = (GPIOB->PCTL & 0xFFFF00FF) | 0x00003300;

    I2C0->MCR = 0x10;                     /* Master function */
    I2C0->MTPR = 7;                       /* ~100 kHz at common 16 MHz bus */
}

void I2C0_WriteByte(unsigned char address, unsigned char data)
{
    I2C0->MSA = (address << 1);
    I2C0->MDR = data;
    I2C0->MCS = 3;                        /* START + RUN */
    I2C0_Wait();
}
```

15. oled.h

```
#ifndef OLED_H
#define OLED_H
void OLED_Init(void);
void OLED_Clear(void);
void OLED_SetCursor(unsigned char row, unsigned char col);
void OLED_PrintString(const char *s);
void OLED_PrintNumber(unsigned short n);
#endif
```

16. oled.c

```
#include "TM4C123GH6PM.h"
#include "delay.h"
#include "i2c.h"
#include "oled.h"
#include "font.h"

#define OLED_ADDR 0x3C

static void OLED_Cmd(unsigned char c)
{
    I2C0_WriteByte(OLED_ADDR, 0x00);
    I2C0_WriteByte(OLED_ADDR, c);
}

static void OLED_Data(unsigned char d)
{
    I2C0_WriteByte(OLED_ADDR, 0x40);
    I2C0_WriteByte(OLED_ADDR, d);
}

void OLED_Init(void)
{
    Delay_ms(100);
    OLED_Cmd(0xAE);
    OLED_Cmd(0xD5); OLED_Cmd(0x80);
    OLED_Cmd(0xA8); OLED_Cmd(0x3F);
    OLED_Cmd(0xD3); OLED_Cmd(0x00);
    OLED_Cmd(0x40);
    OLED_Cmd(0x8D); OLED_Cmd(0x14);
    OLED_Cmd(0x20); OLED_Cmd(0x00);
    OLED_Cmd(0xA1);
    OLED_Cmd(0xC8);
    OLED_Cmd(0xDA); OLED_Cmd(0x12);
    OLED_Cmd(0x81); OLED_Cmd(0x7F);
    OLED_Cmd(0xD9); OLED_Cmd(0xF1);
    OLED_Cmd(0xDB); OLED_Cmd(0x40);
    OLED_Cmd(0xA4);
    OLED_Cmd(0xA6);
    OLED_Cmd(0xAF);
    OLED_Clear();
}

void OLED_Clear(void)
{
    unsigned char page, col;
    for(page=0; page<8; page++)
    {
        OLED_Cmd(0xB0 + page);
        OLED_Cmd(0x00);
        OLED_Cmd(0x10);
        for(col=0; col<128; col++)
            OLED_Data(0x00);
    }
}

void OLED_SetCursor(unsigned char row, unsigned char col)
{
    OLED_Cmd(0xB0 + row);
    OLED_Cmd(col & 0x0F);
    OLED_Cmd(0x10 | ((col >> 4) & 0x0F));
}

void OLED_PrintString(const char *s)
{
    unsigned char p[5];
    unsigned char i;
    while(*s)
    {
        Font_Get((unsigned char)*s, p);
        for(i=0;i<5;i++) OLED_Data(p[i]);
        OLED_Data(0x00);
        s++;
    }
```

```
}

void OLED_PrintNumber(unsigned short n)
{
    char buf[6];
    int i=0, j;
    if(n==0) { OLED_PrintString("0"); return; }
    while(n>0 && i<5)
    {
        buf[i++] = (char)('0' + n%10);
        n/=10;
    }
    for(j=i-1;j>=0;j--)
    {
        char s[2];
        s[0]=buf[j]; s[1]='\0';
        OLED_PrintString(s);
    }
}
```

17. font.h

```
#ifndef FONT_H
#define FONT_H
void Font_Get(unsigned char c, unsigned char *p);
#endif
```

18. font.c

```
#include "font.h"
#include <string.h>

static const unsigned char blank[5]={0,0,0,0,0};

static const unsigned char digits[10][5]={
{0x3E,0x51,0x49,0x45,0x3E},{0x00,0x42,0x7F,0x40,0x00},
{0x42,0x61,0x51,0x49,0x46},{0x21,0x41,0x45,0x4B,0x31},
{0x18,0x14,0x12,0x7F,0x10},{0x27,0x45,0x45,0x45,0x39},
{0x3C,0x4A,0x49,0x49,0x30},{0x01,0x71,0x09,0x05,0x03},
{0x36,0x49,0x49,0x49,0x36},{0x06,0x49,0x49,0x29,0x1E}};

void Font_Get(unsigned char c, unsigned char *p)
{
    unsigned char i;
    if(c >= '0' && c <= '9')
    {
        for(i=0;i<5;i++) p[i]=digits[c-'0'][i];
    }
    else if(c==' ') for(i=0;i<5;i++) p[i]=0;
    else if(c==':') {p[0]=0;p[1]=0x36;p[2]=0;p[3]=0x36;p[4]=0;}
    else if(c=='!') {p[0]=0;p[1]=0;p[2]=0x5F;p[3]=0;p[4]=0;}
    else if(c=='-') {p[0]=0;p[1]=0x08;p[2]=0x08;p[3]=0x08;p[4]=0;}
    else if(c=='A') {p[0]=0x7E;p[1]=0x11;p[2]=0x11;p[3]=0x11;p[4]=0x7E;}
    else if(c=='B') {p[0]=0x7F;p[1]=0x49;p[2]=0x49;p[3]=0x49;p[4]=0x36;}
    else if(c=='C') {p[0]=0x3E;p[1]=0x41;p[2]=0x41;p[3]=0x41;p[4]=0x22;}
    else if(c=='D') {p[0]=0x7F;p[1]=0x41;p[2]=0x41;p[3]=0x22;p[4]=0x1C;}
    else if(c=='E') {p[0]=0x7F;p[1]=0x49;p[2]=0x49;p[3]=0x49;p[4]=0x41;}
    else if(c=='F') {p[0]=0x7F;p[1]=0x09;p[2]=0x09;p[3]=0x09;p[4]=0x01;}
    else if(c=='G') {p[0]=0x3E;p[1]=0x41;p[2]=0x49;p[3]=0x49;p[4]=0x7A;}
    else if(c=='H') {p[0]=0x7F;p[1]=0x08;p[2]=0x08;p[3]=0x08;p[4]=0x7F;}
    else if(c=='I') {p[0]=0;p[1]=0x41;p[2]=0x7F;p[3]=0x41;p[4]=0;}
    else if(c=='L') {p[0]=0x7F;p[1]=0x40;p[2]=0x40;p[3]=0x40;p[4]=0x40;}
    else if(c=='M') {p[0]=0x7F;p[1]=0x02;p[2]=0x0C;p[3]=0x02;p[4]=0x7F;}
    else if(c=='N') {p[0]=0x7F;p[1]=0x04;p[2]=0x08;p[3]=0x10;p[4]=0x7F;}
    else if(c=='O') {p[0]=0x3E;p[1]=0x41;p[2]=0x41;p[3]=0x41;p[4]=0x3E;}
    else if(c=='P') {p[0]=0x7F;p[1]=0x09;p[2]=0x09;p[3]=0x09;p[4]=0x06;}
    else if(c=='R') {p[0]=0x7F;p[1]=0x09;p[2]=0x19;p[3]=0x29;p[4]=0x46;}
    else if(c=='S') {p[0]=0x46;p[1]=0x49;p[2]=0x49;p[3]=0x49;p[4]=0x31;}
    else if(c=='T') {p[0]=0x01;p[1]=0x01;p[2]=0x7F;p[3]=0x01;p[4]=0x01;}
    else if(c=='U') {p[0]=0x3F;p[1]=0x40;p[2]=0x40;p[3]=0x40;p[4]=0x3F;}
    else if(c=='V') {p[0]=0x1F;p[1]=0x20;p[2]=0x40;p[3]=0x20;p[4]=0x1F;}
    else if(c=='W') {p[0]=0x3F;p[1]=0x40;p[2]=0x38;p[3]=0x40;p[4]=0x3F;}
    else if(c=='Y') {p[0]=0x07;p[1]=0x08;p[2]=0x70;p[3]=0x08;p[4]=0x07;}
    else memcpy(p,blank,5);
}
```

19. led.h

```
#ifndef LED_H
#define LED_H
void LED_Init(void);
void LED_On(void);
void LED_Off(void);
#endif
```

20. led.c

```
#include "TM4C123GH6PM.h"
#include "led.h"

void LED_Init(void)
{
    SYSCTL->RCGCGPIO |= (1U << 1);       /* Port B */
    while((SYSCTL->PRGPIO & (1U << 1)) == 0);
    GPIOB->DIR |= (1U << 4);             /* PB4 */
    GPIOB->DEN |= (1U << 4);
    GPIOB->DATA &= ~(1U << 4);
}

void LED_On(void)  { GPIOB->DATA |=  (1U << 4); }
void LED_Off(void) { GPIOB->DATA &= ~(1U << 4); }
```

21. buzzer.h

```
#ifndef BUZZER_H
#define BUZZER_H
void Buzzer_Init(void);
void Buzzer_On(void);
void Buzzer_Off(void);
#endif
```

22. buzzer.c

```
#include "TM4C123GH6PM.h"
#include "buzzer.h"

void Buzzer_Init(void)
{
    SYSCTL->RCGCGPIO |= (1U << 1);       /* Port B */
    while((SYSCTL->PRGPIO & (1U << 1)) == 0);
    GPIOB->DIR |= (1U << 5);             /* PB5 */
    GPIOB->DEN |= (1U << 5);
    GPIOB->DATA &= ~(1U << 5);
}

void Buzzer_On(void)  { GPIOB->DATA |=  (1U << 5); }
void Buzzer_Off(void) { GPIOB->DATA &= ~(1U << 5); }
```

23. ultrasonic.h

```
#ifndef ULTRASONIC_H
#define ULTRASONIC_H
void Ultrasonic_Init(void);
unsigned short Ultrasonic_GetDistanceCM(void);
#endif
```

24. ultrasonic.c

```
#include "TM4C123GH6PM.h"
#include "delay.h"
#include "ultrasonic.h"

static void TriggerPulse(void)
{
    GPIOB->DATA &= ~(1U << 6);
    Delay_us(2);
    GPIOB->DATA |= (1U << 6);
    Delay_us(10);
    GPIOB->DATA &= ~(1U << 6);
}

void Ultrasonic_Init(void)
```

```
{
    SYSCTL->RCGCGPIO |= (1U << 1);
    while((SYSCTL->PRGPIO & (1U << 1)) == 0);

    GPIOB->DIR |= (1U << 6);
    GPIOB->DIR &= ~(1U << 7);
    GPIOB->DEN |= (1U << 6) | (1U << 7);
    GPIOB->DATA &= ~(1U << 6);
}

unsigned short Ultrasonic_GetDistanceCM(void)
{
    unsigned long timeout;
    unsigned long count;

    TriggerPulse();

    timeout = 60000;
    while((GPIOB->DATA & (1U << 7)) == 0)
    {
        if(timeout-- == 0) return 0xFFFF;
        Delay_us(1);
    }

    count = 0;
    while(GPIOB->DATA & (1U << 7))
    {
        if(count++ >= 30000) return 0xFFFF;
        Delay_us(1);
    }

    return (unsigned short)(count / 58);
}
```

25. KEIL PROJECT SETUP

- Add all .c files to the Keil project and all .h files to the project include path.
- Use the TM4C123GH6PM device startup/system file supplied with your Keil project.
- Do not add a second startup file.
- Build once after adding the files and resolve any device-header/version mismatch before connecting the complete hardware.
- The ADC code assumes PE0–PE3 are used as AIN3–AIN0. Verify the TM4C123 datasheet/pinout and your exact LaunchPad revision before wiring.
- The OLED code assumes a 128×64 SSD1306 at I²C address 0x3C.
- The distance thresholds are prototype thresholds and must be calibrated experimentally for the 35 cm test box.

26. TESTING ORDER — DO THIS SEQUENTIALLY

8. Test only the TM4C123 + LED. Confirm PB4 toggles correctly.
9. Test only the buzzer. Confirm it is OFF at startup and turns ON only when commanded.
10. Test the OLED using I²C. Confirm text appears.
11. Connect ONE analog sensor to its assigned ADC channel and verify readings.
12. Repeat for all four sensors, one at a time.
13. Connect the four sensor outputs and verify that the four ADC readings change independently.
14. Connect HC-SR04 using the ECHO divider and test distance separately.
15. Only after individual tests pass, combine the complete system.
16. Finally perform the 360° RC-car test in the fog/cardboard test box.

27. EXPECTED FINAL BEHAVIOUR

Condition	OLED	LED	Buzzer
> 30 cm	MONITOR	OFF	OFF
20–30 cm	DETECTED	ON	OFF

10–20 cm	APPROACH	ON	OFF
≤ 10 cm	CRITICAL	ON	ON
No echo	NO ECHO	OFF	OFF

Note: the 20/30/10 cm values are engineering test thresholds for the prototype, not a validated real-road safety standard. Calibrate them against the actual 35 cm test enclosure.

28. IMPORTANT LIMITATION

The four analog sensors are used for acoustic/pressure-wave sensing and directional comparison. They do not directly provide an exact distance measurement. The HC-SR04, if used, provides supporting proximity information. The claim that acoustic sensing occurs before visual detection must be experimentally evaluated by comparing acoustic detection time with the chosen visual-detection reference.

29. FINAL CONNECTION CHECKLIST

□ All four sensors have a common 3.3V supply.

□ All five breadboards share common GND.

□ Four sensor outputs go to PE3, PE2, PE1, PE0 as configured.

□ OLED SDA = PB3 and SCL = PB2.

□ LED has a 220–330 Ω series resistor.

□ Buzzer polarity is correct and is OFF at startup.

□ HC-SR04 is powered from 5V/VBUS.

□ HC-SR04 ECHO is level-shifted before PB7.

□ No 5V signal is connected directly to a TM4C123 GPIO/ADC input.

□ Sensor model and pinout have been verified from the actual module before power-up.

END OF CONNECTION & CODE GUIDE